Uncertainty-Aware Scan-to-BIM

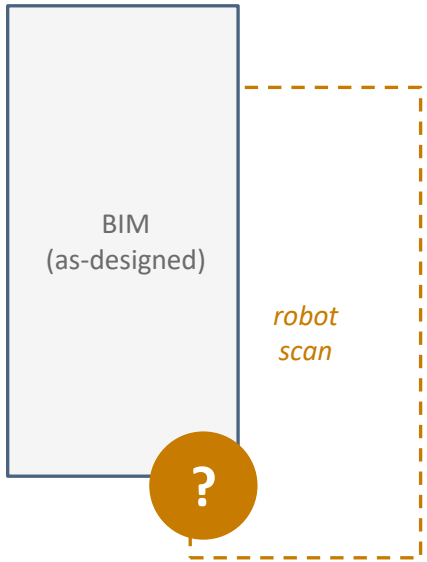
via Evidential Deep Learning

A Calibrated Decision Layer for Autonomous IFC Updates
in Robot-Mounted Digital-Twin Maintenance

Robert Ashe • CONE 652 • Prof. Mani Amani • May 2026

A robot can detect that something has changed — but can't decide what to do about it.

- Manual Scan-to-BIM: 2–4 weeks, ~$10K+ per medium-sized building.
- Robots can scan and create BIMs autonomously, but without quantifying confidence *and* uncertainty, there's no way to disambiguate real changes from sensor noise.
- No principled way to say "I'm sure — go ahead and edit" vs. "I'm not sure — ask a human."
- Existing systems use fixed geometric thresholds to filter updates.
- **No quantified uncertainty or perception confidence in current systems.**



Offset detected by robot — but can we trust it?

Three steps in the Scan-to-BIM workflow are ripe for automation. *The third has no good answer yet.*

Step	Status	Evidence
1. Per-point semantic classification of scan	✓ Solved	Modern segmentation networks: 70%+ mIoU, 96–99% element-level precision
2. Per-element discrepancy detection vs. BIM	✓ Solved	Hu & Brilakis (2024) and follow-on work — mature geometric matching
3. Decide which discrepancies warrant a BIM edit	**? Open**	**No principled formalization.** *The gap in the existing literature this proposal seeks to fill.*

Z. Hu and I. Brilakis, “Matching design-intentplanar, curved, and linear structural instances in point clouds,” Automation in Construction, vol. 158, p. 105219, 2024.

This isn't another EDL paper — it’s about solving the IFC update problem.

Work	Uncertainty Quantification	Domain	BIM (IFC) Update?
BIM2RDT (Akhavian 2025)	None	Indoor + outdoor	Events only
Vassilev 2024	Bayesian sampling (multi-pass)	Bridges	No
EvLPSNet, ULOPS	Evidential	Outdoor LiDAR (autonomous vehicles)	No
This work	**Evidential (single-pass)**	**Indoor / outdoor / BIM**	**Yes (decision policy)**

Method
single-pass evidential deep learning
vs. Bayesian multi-pass sampling

Domain
indoor / outdoor BIM
vs. outdoor / bridges

Downstream coupling
decision policy mapping
vs. terminal output

What the heck is Evidential Deep Learning?
EDL doesn't predict probabilities — it quantifies evidence *and* confidence.

Standard softmax

Is this a cat?

- Outputs a single probability per class.
- "60%" is reported the same whether it has seen lots of evidence or none at all.
- **Cannot distinguish ambiguity from ignorance.**
  - Ambiguity: *that lion looks a lot like a cat*
  - Ignorance: *well, it's not a dog, so I'll guess cat*

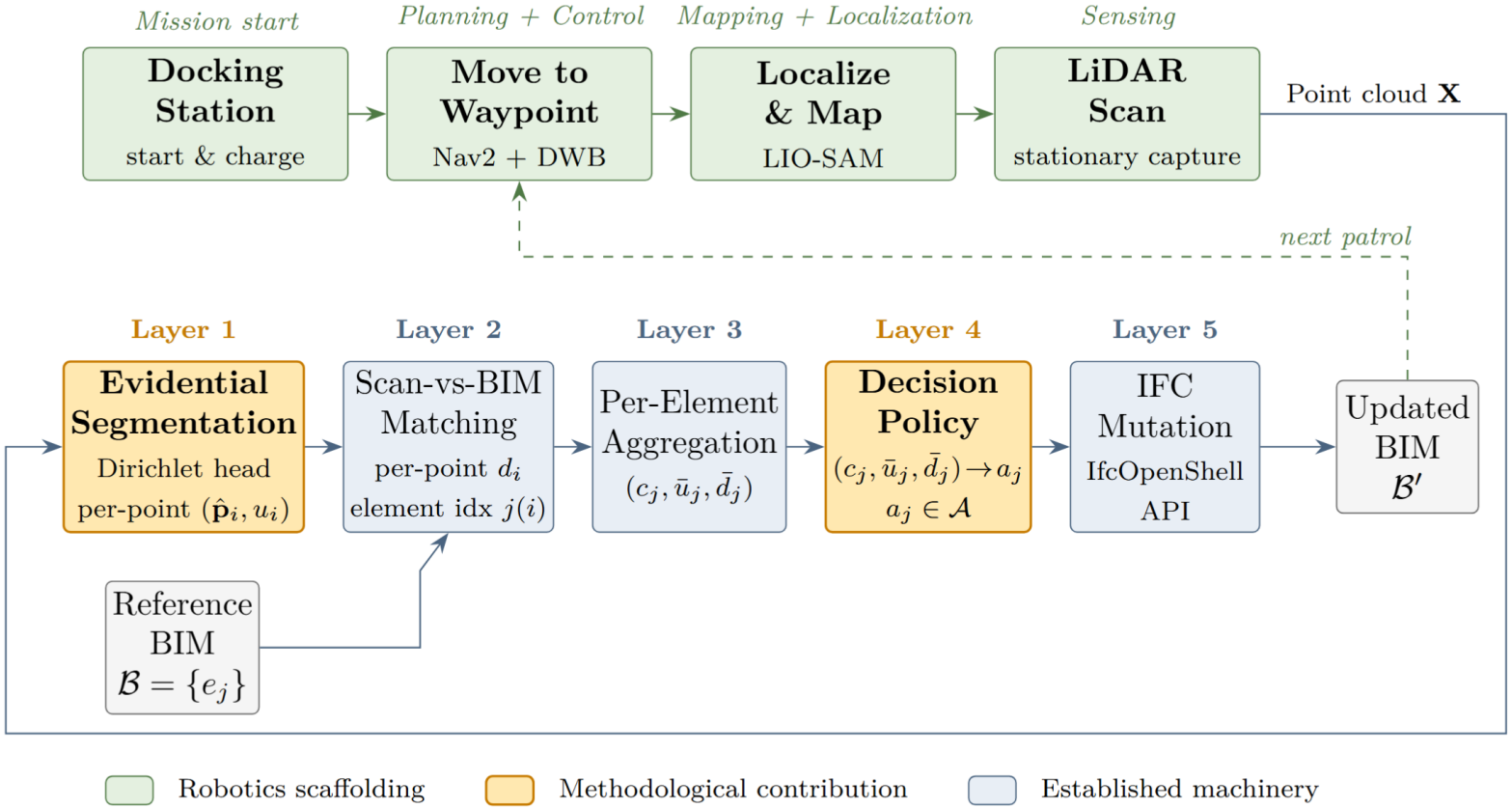
Evidential Deep Learning (EDL)

Is this a cat?

- Outputs total evidence to produce predictive probability and uncertainty per class.
- **Distinguishes ambiguity:** *cat ears, cat tail, huge mane → not a cat*
- **Honest about ignorance:** *not a dog, but might not be a cat, either*

Single forward pass. Bayesian alternatives (multi-class dropout, ensembles) need 3–20 passes for the same confidence signal — a non-starter on a robot (real-time, limited resources).

The system has two layers: quadruped robot scaffolding (top) and EDL perception/decision pipeline (bottom).



Four pillars, four off-the-shelf solutions, one clean integration interface.

Mapping

2D occupancy grid for navigation + per-patrol 3d point clouds registered to BIM frame for perception

via LIO-SAM: LiDAR Inertial Odometry via Smoothing and Mapping

Localization

Continuous odometry + loop closure + kidnapped robot recovery

via LIO-SAM, Scan Context (loop closure detection), SG-ICP (fallback point cloud registration)

Planning

Global path planning + local obstacle avoidance

via ROS 2 Nav2, A*, DWB (*Dynamic Window-Approach for ROS2)*

Control

Velocity-command interface; gait & balance handled onboard

via platform vendor SDK (Boston Dynamics Spot, Unitree Go1/Go2)

Integration interface: robot delivers LiDAR point clouds for localization+mapping and EDL perception layer to perform autonomous BIM updates.

Three per-element signals drive the decision.

Inputs

- Robot-acquired (LiDAR) point cloud $\mathbf{X} = \{\mathbf{p_1}, \ldots, \mathbf{p_N}\}$
- IFC-format BIM elements $\boldsymbol{\mathcal{B}} = \{\mathbf{e_1}, \ldots, \mathbf{e_M}\}$

Per-point $(\hat{p}_i, u_i)$ outputs – (Layer 1, EDL)

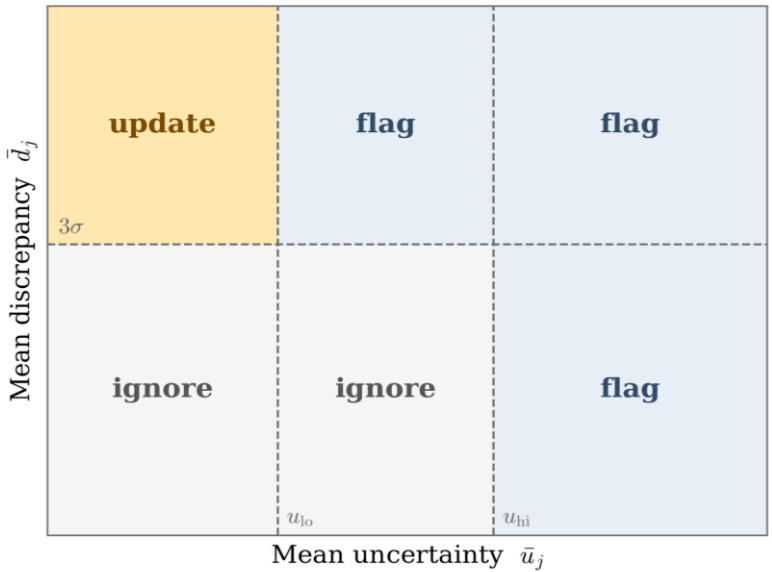
- Class probability $\hat{\mathbf{p}}_{\mathbf{i,k}} = \boldsymbol{\alpha}_{\mathbf{i,k}} / \boldsymbol{\alpha}_{\mathbf{i,0}}$
- Uncertainty $\mathbf{u_i} = \mathbf{K} / \boldsymbol{\alpha}_{\mathbf{i,0}}$

Per-element $(c_j, \bar{u}_j, \bar{d}_j)$ signals – (Layer 3 aggregates)

- Coverage $\boldsymbol{c_j}$ — *did the robot see this element?*
- Mean uncertainty $\bar{\boldsymbol{u}}_{\boldsymbol{j}}$ — *did the network know what it was looking at?*
- Mean discrepancy $\bar{\boldsymbol{d}}_{\boldsymbol{j}}$ — *is it where the BIM says it should be?*

Decision rule: the only action that mutates the BIM is *update*, and it requires **both** low $\bar{u}_j$ **and** a discrepancy above scanner noise. Ambiguous cases default to *flag-for-review*.

Decision regions in the (uncertainty, discrepancy) plane



Coverage-gated: if $c_j < c_{\min}$, the element is marked missing before the (u, d) lookup runs.

EDL training objective: predict well and admit uncertainty when it's ambiguous.

$$\mathcal{L}_{\text{EDL}} = \sum_k \left[(y_k - \hat{p}_k)^2 + \frac{\hat{p}_k(1-\hat{p}_k)}{\alpha_0 + 1} \right] + \lambda_t \, \text{KL}\big(\text{Dir}(\tilde{\boldsymbol{\alpha}}) \,\|\, \text{Dir}(1) \big)$$

predict the right class | *admit ignorance off-manifold*

- First term — drive the predicted probability toward the correct class.
- KL term — penalize over-confident evidence on inputs the network shouldn't be confident about.
- λ_t annealing — ramp KL weight over the first 10 epochs (mandatory to avoid premature convergence).

Constraint	Purpose
$\boldsymbol{d_{translate} > 3\sigma_{range}}$	Discrepancy must exceed scanner noise floor
$\boldsymbol{u_{lo} < u_{hi}}$	Uncertainty thresholds well-separated
$\textbf{default action} = \boldsymbol{flag\_for\_review}$	Conservative-action principle (undefined inputs)

M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential deep learning to quantify classification uncertainty," in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.

Six distinct sources of uncertainty — the decision policy is what holds them together.

Label used in proposal	What it is	How it's handled
U1 - Aleatoric (data)	Inherent ambiguity at class boundaries (cats that look like lions)	EDL predictive entropy (objective function)
U2 - Epistemic (model)	Out-of-distribution / novel inputs (hairless cats / ugly dogs)	EDL uncertainty $u_i = K/\alpha_o$ — primary signal
U3 - Sensor noise	**LiDAR range error, registration error (limitation of sensor itself)**	**Threshold $d_{translate} > 3\sigma_{range}$** (discrepancy threshold > LiDAR sensitivity)
U4 - Coverage estimation	Expected point count is itself estimated (not sure how many BIM elements there are)	Conservative bias toward `mark-missing` action
U5 - Decision threshold calibration	Decision-policy thresholds are estimates until verified in PoC	Sensitivity analysis (PoC deliverable)
U6 - BIM ground truth	Reference BIM treated as oracle	Acknowledged limitation; open question; not formally modeled

U1, U2 — EDL's domain (primary)

U3, U4, U5 — handled by other mechanisms

U6 — acknowledged open limit

Two-stage proof of concept.

Baseline stage (establish core models)

1. Environment bootstrap (PyTorch 2.3 + CUDA 12.1, RTX 3070 eGPU, 8 GB VRAM)
2. PointNet++ baseline reproduction (target ≥ 55% S3DIS Area-5 mIoU)
3. PointMetaBase-L training, then evidential head replacement (within 1–2 mIoU of softmax)
4. **Calibration table: 5 UQ methods × 5 metrics + inference time**

Synthetic Scan-vs-BIM stage (full pipeline verification)

1. HELIOS++ synthetic Scan-vs-BIM scenario (one element displaced ≥ 30 cm)
2. Run trained EDL model on synthetic scan, compute per-element signals
3. Apply decision policy → verify update / flag fires correctly
4. Sensitivity sweep across all four thresholds

~30-days

PoC budget

compute + engineering time

Fallback

If HELIOS++ blows the budget → use SPC dataset with manually corrupted element.

Hardware	RTX 3070 eGPU (8 GB VRAM, Thunderbolt 5) · Intel Core Ultra 9 285H + 64 GB RAM · Ubuntu 24.04
Software	PyTorch + PyG · Open3D + IfcOpenShell · HELIOS++ · Bonsai/BlenderBIM

What I’m proposing, in simple terms.

Uncertainty must be accounted for to create trustworthy digital twins.

Evidential deep learning quantifies uncertainty, giving the robot a reliable measure of perception confidence so it knows when to autonomously update the BIM and when to defer to a human.

Future work, in priority order

1. Closed-loop IFC mutation (live BIM editing + re-scan validation)
2. Cross-dataset generalization (ScanNet, BIMNet)
3. Calibration of decision-policy thresholds
4. Probability distribution over reference BIM (the open frontier)

Questions

Robert Ashe • CONE 652 • rashe7414@sdsu.edu

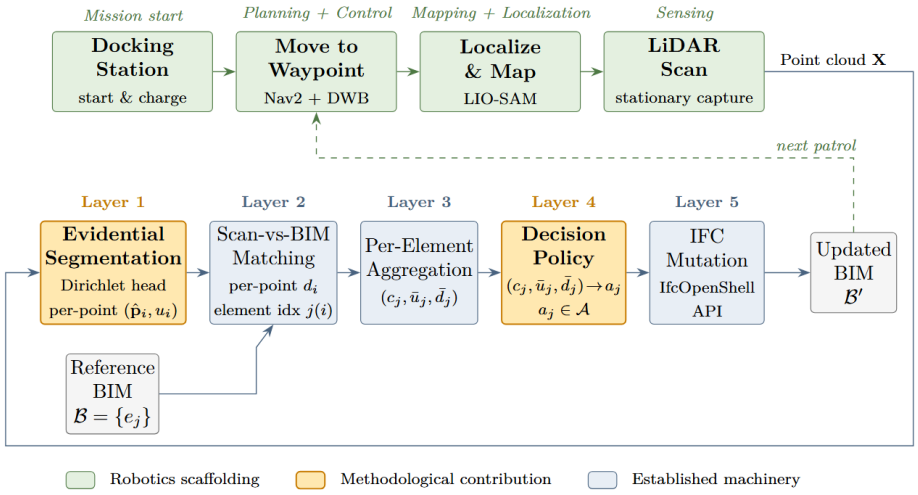


Figure 1 — system architecture